

iOS DEVELOPMENT 1

Week 1

Toolchain and Swift Fundamentals

```
let course = "iOS Development 1"  
var week = 1
```

Today

- 1 What this course is, and how it is graded
- 2 Why UIKit and not SwiftUI
- 3 Xcode and what the files in a project do
- 4 Swift: constants, types, strings, collections, control flow, functions
- 5 The ten things that will trip you up coming from Java

How you are graded

Labs (12, lowest dropped)

20%

Midterm — week 7

20%

Endterm — week 14

20%

Final project — week 15

40%

30%

of every lab mark from week 4 is design

Responsibility separation, naming, coupling, choosing the right type. Code that works and is badly structured is a C.

Rules that do not bend

1

Xcode version is pinned

Do not upgrade mid-semester. Apple ships in September. An upgrade during exam period will break your project.

2

Programmatic UI only

No storyboards in graded work. You will see one, once, in week 5.

3

Everything is in git

From today. Weeks 8+ require real commit history, not one dump at the deadline.

Every lab from week 4

20 minutes

of each lab, a real submission goes on screen and the room critiques it against one named criterion.

Announced now, not in week 4, so you can plan for it.

Responsibility

Naming

Coupling

Testability

Extension points

One criterion per week, rotating.

Why UIKit, when SwiftUI exists

What you gain

- The platform is visible — lifecycle, memory and delegation are explicit
- Most production code you will meet is UIKit or hybrid
- Next semester is about what happens underneath. That is far easier to see here

What you give up

- SwiftUI is not taught in this course
- You will meet it in internships and interviews
- Week 14 gives you a map and where to read next

A deliberate trade: depth in one paradigm over shallow coverage of two.

The window

Navigator

left — your files

Editor

centre — where you type

Inspectors

right — settings for the selection

Debug area

bottom — print output lands here

Scheme + destination

top — what runs, and where

Edit → run → observe. That loop is the only thing you must leave this segment with.

What the files actually do

AppDelegate

Process-level events — launch, termination

SceneDelegate

Window setup and UI lifecycle

ViewController

One screen

Info.plist

Configuration the system reads before your code runs

Assets

Images, colours, app icon

Templates differ between Xcode versions. Online tutorials will not match yours exactly.

Learn the language here first

```
let greeting = "Hello"  
let count = 3  
  
for i in 1...count {  
    print(greeting)  
}
```

No build cycle.
Results appear as you type.

Use playgrounds for weeks 1 to 4.
Once we reach UIKit in week 5, they stop being useful.

PART TWO

Swift fundamentals

You know Java. Most of this will look familiar. The parts that do not are the parts that matter.

let and var

```
let name = "Aisha"    // constant
var score = 0          // variable

score = 10             // fine
name = "Bolot"        // compile error
```

Default to let

Use var only when you have a reason. This is the opposite of the habit you have.

Xcode warns you when a var never changes.

The compiler works out the type

```
let count = 42           // Int
let price = 9.99         // Double
let title: String = "iOS" // explicit when you want it
let ratio: Double = 5    // explicit changes what you get
```

Inference is not dynamic typing. Every type is fixed at compile time — you just did not have to write it.

Swift will not convert for you

```
let a = 5
let b = 2.0

let c = a + b
// error
```

```
let a = 5
let b = 2.0

let c = Double(a) + b
// 7.0
```

Java widens int to double silently. Swift refuses. Every conversion is written by you, on purpose — this will annoy you for two weeks and then start saving you.

There are no primitives

```
let n = 5
```

```
n.isMultiple(of: 2) // false
```

```
n.description      // "5"
```

Int is a struct with methods.

There is no int / Integer split, no boxing, no wrapper classes to remember.

Strings and interpolation

```
let first = "Arman"
var score = 10

let line = "Hello, \$(first). You have \$(score) points."

let multi = """
Multi-line strings
work like this.
"""
```

Interpolation replaces concatenation. You will almost never write + between strings again.

A copy is a real copy

```
var x = "hello"  
var y = x  
  
y.append(" world")  
  
print(x)    // "hello"  
print(y)    // "hello world"
```

In Java the same reasoning happens for a different reason — String is immutable.

Here, y is genuinely a separate value.

The difference matters on the next slide.

Arrays are values, not references

```
var numbers = [1, 2, 3]
var copy = numbers

copy.append(4)

print(numbers)  // [1, 2, 3]
print(copy)     // [1, 2, 3, 4]
```

```
let fixed = [1, 2, 3]
fixed.append(4)  // error - let locks the contents too
```

In Java

this prints the same array twice.

This single difference is the root of value semantics in week 2, and of the memory model in week 10.

Dictionaries

```
var ages = ["Ann": 30, "Bolot": 25]

ages["Chen"] = 41

print(ages["Ann"])    // Optional(30)
```

Optional?

Ignore that word this week.

Next lecture it gets 90 minutes, and it is the most important idea in the language.

if and for

```
if score > 5 {  
    print("pass")  
}
```

```
for i in 0..<3 { }    // 0 1 2  
for i in 0...3 { }    // 0 1 2 3
```

```
for n in numbers { }
```

Two differences

- No parentheses around the condition
- Braces are mandatory, even for one line
- `..< excludes the upper bound, ... includes it`

switch is stricter and more useful

```
switch score {  
  case 0:  
    print("none")  
  case 1..<5:  
    print("low")  
  case 5...:  
    print("high")  
  default:  
    print("negative")  
}
```

- No fall-through — you never write break
- Must be exhaustive, or it will not compile
- Cases can match ranges, not just values
- In week 4 it matches enums with data attached, and becomes the reason to use switch at all

Functions

```
func greet(name: String) -> String {  
    return "Hello, \(name)"  
}  
  
greet(name: "Aisha")    // the label is required at the call site
```

The parameter name is part of how the function is called, not just an internal detail.

Argument labels are not decoration

```
func move(from a: Int,  
          to b: Int) { }
```

```
move(from: 0, to: 5)
```

```
func double(_ v: Int) -> Int {  
    v * 2  
}
```

move(from: 0, to: 5)

reads correctly. move(0, 5) does not.

Swift naming is judged at the call site, not the declaration.

This is graded.

The underscore removes the label. Use it only when the argument needs no explanation.

Tuples

```
let point = (x: 3, y: 7)
print(point.x)
```

```
func bounds(_ v: [Int]) -> (min: Int, max: Int) {
    // ...
}
```

Java has no equivalent.

Good for two or three related values. Past that you need a real type — which is week 4.

COMING FROM JAVA

The trap list, part one

You expect

int and double are primitives

int widens to double silently

Arrays are references

String immutable by class design

Variables by default

Swift actually

Everything is a type with methods

Explicit conversion, always

Arrays are values — assignment copies

String is a value type

let by default

The trap list, part two

You expect

switch falls through

switch can be partial

if (x > 5)

foo(a, b)

null

Swift actually

It does not. No break

Must be exhaustive

if x > 5 — no parens, braces required

foo(from: a, to: b) — labels are the name

Optionals — next lecture

NEXT 90 MINUTES

Today's lab

Design workshop

No Swift. No Xcode.

You get a set of requirements and decide what types should exist: what is a class, what is a struct, what is an enum, what does not need to exist at all.

We compare solutions out loud.

Then: a short diagnostic

Ungraded. It does not affect your mark and it does not sort you into a group.

It tells me what the room already knows, and tells you privately where you stand.

Answer honestly — a flattering result helps nobody.

Before next lecture

1

Xcode installed at the pinned version, a project that builds, screenshot committed to your repo

2

A playground with every construct from today, typed by hand — not copied

3

One question about something that surprised you today

Install problems are normal and they are not your fault. Post them on the forum — the first 24 hours generate more support requests than any other week.

NEXT LECTURE

Optionals, value semantics, and Swift's class model

Two-phase initialisation, deinit, access control, final. The hardest lecture of the first block — come having typed this week's code.

```
var maybe: String? = nil    // see you Tuesday
```